TECHNICAL DEEP-DIVE · TEAM MOLECULENS

Chicago Crime Branch
How we built it, step by step

Tech stack · missing data · data cleaning · data analysis · how every metric is derived

Every number in this document comes from the project's own output files (cleaning_log.json, kpis.json, insights.json, methodology.json).

Contents

1. Tech stack in detail (what, why, where used)
2. The big picture: architecture and data flow
3. Step-by-step data cleaning (including all missing-data handling)
4. Missing-data policy: one table for every column
5. Step-by-step data analysis process
6. How each handout KPI (K1–K8) is derived
7. How each handout chart (Q1–Q10) is derived
8. How each deeper metric is derived (formulas and methods)
9. How results become the website (export, cube, filters)
10. How we proved the numbers are right
11. Limitations, in plain words

1. Tech stack in detail

1.1 Python side (all the analysis)

Tool (version)	What it is	What we used it for	Why this and not something else
Python 3.13	The programming language	Every calculation, cleaning step and model	The TECHVERSE rulebook says **"Teams must use Python for their analysis."**
pandas 2.3	Table (spreadsheet-like) library	Loading, cleaning, grouping, counting 1,048,575 rows (`groupby`, `crosstab`, `pd.cut`, `to_datetime`)	Standard for data analysis; fast on millions of rows
NumPy	Fast number arrays	Maths inside statistics (percentiles, random sampling, sorting)	Underlies pandas and SciPy

openpyxl	Excel reader	Reads the 150 MB .xlsx in **read-only mode** so the source is never changed	pandas' own reader was slower and loaded everything into memory
PyArrow 23 (parquet)	Compact column file format	Caches the loaded data (`raw.parquet`) and saves the cleaned data (`crimes_clean.parquet`); loading takes seconds instead of ~4 minutes	Much faster than CSV and keeps data types
SciPy 1.17	Scientific/statistics library	chi-square, Kruskal–Wallis, Poisson and chi² distributions (for exact confidence ranges)	Reliable, well-tested statistics
scikit-learn 1.8	Machine-learning library	Logistic regression, gradient boosting (`HistGradientBoostingClassifier`), k-means, silhouette score, Poisson regression, permutation importance, AUC/Brier metrics	Standard, explainable models; allowed by the rulebook ("scikit-learn, where relevant")
Pydantic 2.12	Data-shape checker	Describes exactly what every exported file must contain (ChartSpec, KPI, Insight, Cube, Manifest) and validates it; also generates JSON Schemas	Stops a broken export from reaching the website
matplotlib 3.10	Python plotting	The "QA contact sheet" (a picture of all key charts) and a Python-only fallback to prove charts render	Rulebook-listed library
hashlib (SHA-256)	Built into Python	Fingerprint of the original Excel file and of every exported file	Proves the source was never modified

Not used on purpose: statsmodels and Plotly were not installed, so we did not depend on them (our own small `statlib.py` implements Wilson, Poisson, Benjamini–Hochberg, bootstrap, Cramér's V).

1.2 Website side (everything you see)

Tool	What it is	What we used it for
React 19 + TypeScript	UI library + typed JavaScript	All screens (cards, tabs, filters, charts, map, chat). TypeScript catches mistakes at build time; the types are even **generated from the Python models**.
Vite	Build tool / dev server	Runs `npm run dev` and produces the production build
Motion	Animation library	Counters, tab slide/blur transitions, bars growing, welcome-page letters, chat window
d3-scale, d3-shape, d3-array, d3-interpolate	Only the *maths* parts of D3	Axis scales, smooth line curves, colour ramps. The charts themselves are **hand-written SVG** so every handout rule (order, sort, labels) is exactly controlled.
HTML Canvas	Pixel drawing surface	The district map (2,387 cells) and the welcome-page city
Fontsource (Bricolage Grotesque, Inter, JetBrains Mono)	Fonts installed as packages	Typography without needing internet

DecompressionStream (browser built-in)	Un-zips files	Opens the compressed data cube (0.66 MB)
Node 22 + npm	JavaScript runtime and package manager	Installs packages, runs scripts
esbuild	Fast bundler	The `verify` script bundles the website's calculator and tests it against Python
Edge/Chrome headless	Browser without a window	Prints the PDF reports

2. The big picture: architecture and data flow

```
Crime Dataset.xlsx (150 MB, read-only, SHA-256 fingerprint saved)
        |  load.py        openpyxl -> pandas -> raw.parquet
        |  clean.py       repair dates, fix values, log every change
        |  features.py    Time of Day, Season, Category, hour, flags
        |  windows.py     find the dense periods from daily counts
        |  kpis.py / charts.py        handout K1-K8, Q1-Q10
        |  stats_time.py / stats_arrest.py / stats_geo.py   deeper analysis
        |  insights.py    sentences built from results (no typed numbers)
        |  pipeline.py    runs all of it, builds the "cube"
        |  export.py      JSON files + manifest + schemas + TypeScript types
        |  validate.py    44 automatic checks
        v
   export/  (JSON)  ---sync--->  dashboard/public/data
        v
   React website: loads JSON + cube, recomputes with filters, draws, Dot answers
```

Why this design? (1) The rulebook wants Python analysis, so all maths lives there. (2) The website never calculates "findings"; it only filters and re-adds counts from the cube. (3) Both sides are checked against each other, so a mistake on either side is caught.

3. Step-by-step data cleaning

Rule we followed: **change as little as possible, log everything, delete nothing.** Every action is counted and saved in `cleaning_log.json`. File: `src/clean.py`, main function `clean()`.

Step 0: Look before touching

- The workbook has 1 sheet, **1,048,575 data rows × 22 columns**. That equals Excel's row limit (1,048,576 including the header), so we realised early the file is a **slice**, not the full table.
- The handout expects K1 = 1,048,575 and 214,846 arrests (20.5%). Our raw counts matched, which told us we had the right file.
- Columns: ID, Case Number, Date, Block, IUCR, Primary Type, Description, Location Description, Arrest, Domestic, Beat, District, Ward, Community Area, FBI Code, X/Y Coordinate, Year, Updated On, Latitude, Longitude, Location.

- Raw missing values found: Location Description **8,337**; Ward **16,478**; Community Area **16,505**; X/Y Coordinate, Latitude, Longitude, Location **327,690** (31.3%). Every other column had no gaps.

Step 1: Load safely and fix mixed types (`load.py → load_raw()`)

- **Problem:** Excel stored some columns as a mix of numbers and text (for example FBI Code has both `14` and `08B`; IUCR similar). Saving to parquet would fail or silently change values.
- **Fix:** every text-like column is converted to text so nothing is lost.
- **Date columns** were mixed too: 623,937 cells were text, 424,638 were real Excel dates. We stored a tag (`Date__src` = "text" or "datetime") **before** converting, because that tag is the key to repairing the dates (Step 2).
- The workbook is opened with `read_only=True` and a **SHA-256 fingerprint, size and modified-time** are saved. At the end of every run the fingerprint is recomputed and compared. It never changed, so the original file is untouched.

Step 2: Repair the dates (the biggest cleaning step)

What we noticed:

- Text cells looked like `07/29/2022 03:39:00 AM` and the day (second number) was **always 13 or higher**.
- Real Excel date cells always had a "day" of **12 or lower**, and their "month" values were spread flat (about 33k each) while their "day" values had the seasonal shape (more in summer).

What this means: the original data is month/day/year. When Excel opened the CSV in a day-first setting, any date that *could* be read both ways (day ≤ 12) was turned into a real date with **month and day swapped**. Dates with day ≥ 13 could not be read day-first, so they stayed as text and were correct.

What we did (`_parse_stamp()`):

1. Text cells: parse with the exact pattern `%m/%d/%Y %I:%M:%S %p` (month/day/year, 12-hour clock).
2. Excel-date cells: rebuild the date with the **month and day exchanged**; hour, minute and second are kept.

How we proved it worked (4 independent tests):

- **Year check:** the repaired date's year equals the separate `Year` column for **100%** of rows (swapping month/day never changes the year, so this confirms nothing else broke).
- **"Updated On" check:** a crime cannot be updated before it happened. Before repair, **3.30%** of rows were dated *after* their own update time. After repair: **0.00%**.
- **Handout check:** the monthly chart then has exactly 28 points, a low of **18,710** in February 2016, and highs of **24,781 (Aug 2015)** and **24,818 (Aug 2016)**, matching the handout's example.
- **Unparseable dates:** 0.

Step 3: Tidy text values

- Primary Type, Description, Location Description and Block: leading/trailing spaces removed and set to upper case, so "Theft " and "THEFT" cannot become two categories.
- **Location Description missing (8,337 rows):** filled with the word **"UNKNOWN"**. We did not guess a place; "UNKNOWN" keeps the row countable and honest.

Step 4: Crime-type labels

- There are **33** raw crime types. Two of them, `CRIM SEXUAL ASSAULT` and `CRIMINAL SEXUAL ASSAULT`, are the same offence under two names (the police renamed it).

- The handout lists **both** as violent and K4 counts "different crime types", so we keep the raw column unchanged for handout numbers (K4 = **33**).
- For deeper analysis we add a second column, `Primary Type Harmonised` (merged, **32** types), so the merged offence is not split in half.

Step 5: Geography and missing locations

- **Coordinates missing: 327,690 rows (31.3%).** We did **not** fill these in (no guessing latitude/longitude). We added a flag `has_coords` and simply draw maps only from rows that have coordinates.
- **Impossible coordinates: 16 rows.** The Chicago box we used is latitude 41.6–42.1 and longitude −87.95 to −87.5. Points outside it (for example latitude 36.6 and longitude −91.7, which is in another state) were set to missing in Latitude, Longitude, X and Y. Total missing coordinates after cleaning: 327,706.
- **Community Area = 0 (2,616 rows):** there is no community area 0 in Chicago, so it is a code for "unknown". We changed it to **missing**. With the 16,505 already missing, community area is missing in **19,121** rows.
- **Ward missing (16,478):** left missing (nullable integer). District and Beat have no gaps, so they are used for the main geographic work.
- **Why missing coordinates matter:** we tested whether they are random. They are not (see Step 9).

Step 6: Duplicates

- Duplicate IDs: **0**. Fully duplicated rows: **0**.
- Repeated Case Numbers: **13**. These have different IDs (several offences under one case), so they are real separate records and we kept them. One row = one crime report, as the handout says.
- The redundant text column `Location` (a copy of lat/long) was dropped as a column only (no rows).

Step 7: Suspicious values we flagged but did NOT remove

- **Placeholder timestamps.** 5.5% of records are stamped exactly 00:00 or 12:00 (41% of all records in the midnight hour and 45% of the noon hour). That is far more than a real clock gives, so many are probably "time unknown" defaults. We added `default_time` = (minute = 0 and hour is 0 or 12).
  - The handout's K7 must use all rows, so the handout number is unchanged (**17.7%**).
  - We also report the version without placeholders (**15.8%**) and exclude them from hour-by-hour analysis.
- **Dates on the 1st of the month.** Sex offence has 14.4% of its records on day 1 (expected about 3.3%), a 4.4× over-representation. We think the true date is often unknown and booked to the 1st. We flag it and explain it as an interpretation, not a proven fact.

Step 8: Feature engineering (`features.py → add_features()`)

New column	How it is made
`hour, month, day, dow, day_of_week, is_weekend, month_start`	Taken from the repaired Date. `month_start` = first day of the month (handout "Month Start").
`time_of_day`	Hour 0–5 = Night, 6–11 = Morning, 12–17 = Afternoon, 18–23 = Evening (handout definition).
`season`	Dec, Jan, Feb = Winter; Mar–May = Spring; Jun–Aug = Summer; Sep–Nov = Fall.

`crime_category`	Violent = Assault, Battery, Robbery, Homicide, Crim Sexual Assault, Criminal Sexual Assault, Kidnapping. Property = Theft, Burglary, Motor Vehicle Theft, Criminal Damage, Arson. Everything else = Other.
`default_time, has_coords`	Quality flags described above.

Step 9: Is the missing data random? (we tested it)

- **Window A (Apr 2015–Jul 2017):** only **1.7%** of records lack coordinates. But they are not random: Sex Offense 13.9%, Offense Involving Children 12.6%, Criminal Sexual Assault 10.1% are missing far more often (likely privacy). Chi-square test with **Cramér's V = 0.23** (moderate link between crime type and missing location).
- **Window B (Oct 2025–Aug 2026): 99.96%** of coordinates are missing. So the map can only use window A.
- **Consequence:** we never impute and never treat the map as the whole truth; the map card says which period it shows.

4. Missing-data policy: every column

Column	Missing in raw	What we did	Why
Date	0 (but 424,638 wrong)	Repaired month/day swap	Needed for every time-based result
Location Description	8,337	Filled with "UNKNOWN"	Keeps the row; does not invent a place
Latitude, Longitude, X, Y	327,690 (+16 invalid)	Left missing; flag `has_coords`; invalid points set to missing	Guessing locations would fabricate data (rulebook forbids it)
Community Area	16,505 (+2,616 zeros)	Zeros converted to missing; left missing	0 is not a real area
Ward	16,478	Left missing	Not needed for any handout metric
All others (ID, Primary Type, Arrest, Domestic, District, Beat, Year …)	0	Nothing needed	Complete

Summary of the policy: no imputation (no averages, no guessed values), no row deletion, no outlier capping. Missing values are either shown as "UNKNOWN", left empty and counted, or flagged. Any analysis that needs a field uses only the rows that have it and says so on screen.

5. Step-by-step data analysis process

1. **Handout layer first.** Compute K1–K8 and Q1–Q10 exactly as defined, on the whole dataset (or the stated period). Check against the handout's known values.
2. **Find reliable periods** (`windows.py → dense_windows()`). Count crimes per day. A "dense window" is a run of at least **60 days** with at least **200 records a day**. Two were found: 2015-03-18 to 2017-08-07 and 2025-09-28 to 2026-09-22. They are trimmed to whole months: **Window A = Apr 2015–Jul 2017 (28 months, 627,787 records)** and **Window B = Oct 2025–Aug 2026 (11 months, 211,787 records)**. **209,001** records lie outside both. This is why deeper analysis never claims multi-year trends.
3. **Data-quality analysis** (missing coordinates, placeholder times, 1st-of-month, coverage grid).
4. **Time analysis** (weekday, hour, season, anomalies, then-vs-now, timing groups).
5. **Arrest analysis** (rates, standardising, district index, models).
6. **Place analysis** (districts, locations, concentration, map grid).
7. **Write the insights** (`insights.py`): each sentence is built from computed values.
8. **Export and validate** (44 checks). Then the website re-checks itself.

Rules we applied to every analysis: (a) report **effect sizes and confidence ranges**, not just p-values (with ~10^5–10^6 rows everything is "significant"); (b) correct for testing many things (Benjamini–Hochberg); (c) test models on **later, unseen months**; (d) fixed random seeds (42) so results repeat exactly; (e) every insight carries a method and a caveat.

6. How each handout KPI is derived

File: `kpis.py → compute_kpis()`. "One crime = one row", so counting = counting rows.

KPI	Formula	Our value	Notes
K1 Total crimes	number of rows	**1,048,575**	Matches handout
K2 Arrest rate	rows with Arrest = TRUE ÷ all rows × 100	214,846 ÷ 1,048,575 = **20.49%**	Matches handout (20.5%)
K3 Domestic %	rows with Domestic = TRUE ÷ all rows × 100	192,310 ÷ 1,048,575 = **18.34%**	
K4 Different crime types	count of distinct Primary Type (raw labels)	**33**	32 if the two sexual-assault labels are merged
K5 Crimes in 2016	rows where Year = 2016	**268,944**	
K6 Average per month	rows with month_start from Apr 2015 to Jul 2017 ÷ 28 months	627,787 ÷ 28 = **22,421**	Uses the "Month Start" column from Q1
K7 Night-time %	rows with hour 0–5 ÷ all rows × 100	185,956 ÷ 1,048,575 = **17.73%**	15.8% without placeholder 00:00/12:00 times
K8 Violent %	rows with crime_category = Violent ÷ all rows × 100	306,138 ÷ 1,048,575 = **29.20%**	Uses the handout's violent list

7. How each handout chart is derived

File: `charts.py → build_handout_charts()`. Each result is a tidy list, already in the order the handout demands.

Chart	Derivation
Q1 Monthly crimes	Group rows by `month_start`, count, keep Apr 2015–Jul 2017 → 28 points (low 18,710 Feb 2016; high 24,818 Aug 2016).
Q2 By weekday	Count by day name, then re-order to Monday…Sunday (not alphabetical).
Q3 By time of day	Count by `time_of_day`, order Night, Morning, Afternoon, Evening.
Q4 By season, 2016	Keep Year = 2016, count by `season`, order Winter, Spring, Summer, Fall.
Q5 Violent/Property/Other share	Count by `crime_category`; share = count ÷ total × 100 (29.2% / 43.3% / 27.6%).
Q6 Arrest rate by time of day	For each time of day: arrests ÷ crimes × 100 (16.2%, 18.2%, 20.6%, 24.6%).
Q7 Top 10 crime types	Count by Primary Type, sort high→low, keep 10 (Theft 236,314 first).
Q8 Top 5 domestic types	Keep Domestic = TRUE, count by type, sort, keep 5 (Battery 100,250 first).
Q9 Top 10 districts + arrest rate	Per district: crimes and arrests; keep the 10 with most crimes (sorted high→low); arrest rate = arrests ÷ crimes. Bars = crimes, line on second axis = arrest rate.
Q10 Weekday × time heat map	Count crimes for each of the 28 (weekday, time-of-day) combinations; the website colours cells with a colour scale.

8. How each deeper metric is derived

8.1 Weekday effect (`stats_time.py → day_of_week_index()`)

- Not every weekday appears the same number of times in a window, so we compare to the **expected** count: expected = total × (days of that weekday ÷ all days).
- **Index** = observed ÷ expected (1.00 = average day). Friday = **1.046** (4.6% above average).
- **Range:** exact Poisson (Garwood) 95% interval from the chi-square distribution.
- **Effect size:** chi-square $\chi^2 = \Sigma(\text{obs} - \text{exp})^2 \div \text{exp}$, then **Cohen's w = $\sqrt{(\chi^2 \div N)}$ = 0.020**. Under 0.1 is "very small", so we say weekday barely matters (even though χ^2 is "significant").

8.2 Hour profile and weekday × hour grid

- Count crimes by hour and by (weekday, hour), **excluding placeholder times**, divided by how many such days exist (crimes per day). Peak hour 18:00; quietest 05:00.
- **Weekend-night ratio:** (sum of Saturday and Sunday 00:00–05:59 per day ÷ 2) ÷ (sum of Mon–Fri 00:00–05:59 per day ÷ 5) = **1.69**.

8.3 Seasonality (`seasonality()`)

- Daily crime counts in window A (853 days). Summer = Jun–Aug average **787.9**/day; winter = Dec–Feb average **674.2**/day.
- **Ratio = 787.9 ÷ 674.2 = 1.169** (about 17% more).

- **Range by bootstrap:** resample the daily counts with replacement 2,000 times, compute the ratio each time, take the 2.5th and 97.5th percentiles → **1.14 to 1.19**.
- **Kruskal–Wallis test** across the four seasons (no assumption of normal data): H = 275.4, p ≈ 10^{-59} (shown only as support).
- **Per crime type** (types with ≥ 3,000 records, 500 bootstrap repeats): Weapons Violation ×1.46 (biggest swing), Deceptive Practice ×0.92 (smallest).

8.4 Daily expectation and anomalies (`daily_glm()`, `anomaly_table()`)

- Model: **Poisson regression** on daily counts with weekday (7), month (12) and US-federal-holiday features.
- **Honest test:** trained on 2015-04 to 2016-12, tested on the next **212 days**. Average error (MAPE) **6.7%** vs **8.2%** for the simple "same weekday last week" guess.
- Then refit on all days. Residual = (actual − expected) ÷ √expected. Robust score z = (residual − median) ÷ (1.4826 × MAD), where MAD is the median absolute deviation (σ ≈ 1.89). Days with |z| > 4 are flagged; 9 of the 12 largest are the 1st of a month (the date artefact).

8.5 First-of-month artefact (`first_of_month_artifact()`)

For each crime type: share on day 1 ÷ (1 ÷ average days in a month ≈ 3.3%) = lift. Sex offence 14.4% → 4.4×. "Excess records" = (share − expected) × records, about 4,800 in window A.

8.6 Then vs now (`era_comparison()`)

- Only calendar months present in both windows are compared (1–8, 10–12; September is missing from window B), so seasons do not bias it.
- Crimes per day: A = **732.8**, B = **632.2**; ratio **0.863** (range 0.851–0.875) → about 14% fewer.
- Per crime type: change % = (per-day B ÷ per-day A − 1) × 100. Motor Vehicle Theft +68%, Prostitution −84%.
- Arrest rate 21.6% → 15.4% (likely understated for recent months because arrests are recorded later).

8.7 Crime-type timing groups (`type_hour_clusters()`)

- For each type (≥ 3,000 records) build a 24-number profile: the share of its crimes in each hour (placeholder times excluded).
- **k-means** clustering for k = 2…6 (20 starts each, seed 42). The best k is chosen by the **silhouette score** (higher = cleaner groups). Result: k = 3, silhouette 0.28, which is weak, so we call the groups descriptive.

8.8 Arrest rate by crime type (`arrest_by_type()`)

Rate = arrests ÷ crimes with a **Wilson 95% interval**: centre = $(p + z^2/2n) \div (1 + z^2/n)$; half-width = $z \cdot \sqrt{(p(1-p)/n + z^2/4n^2)} \div (1 + z^2/n)$, z = 1.96. Wilson is better than the simple formula for rates near 0% or 100%.

8.9 Fair arrest comparison by time of day (`tod_standardised()`) — "direct standardisation"

1. Compute each crime type's share of all crimes (the overall mix).
2. For each time of day, compute the arrest rate *within each crime type*.
3. Standardised rate = Σ (rate in that type × overall share of that type).

Result: raw Night 18.1% → Evening 25.4% (gap **7.3** points); standardised 21.4% → 22.4% (gap **1.0** point). So **86%** of the gap was crime mix. Narcotics make up 2.3% of night crimes but 8.1% of evening crimes and 98.9% of narcotics records end in an arrest.

8.10 District arrest "index" (`district_sir()`) — "indirect standardisation"

1. City-wide arrest rate for each crime type (e.g. narcotics 98.9%).
2. For each district, **expected arrests** = Σ over its crimes of the city rate for that crime's type.
3. **Index (SIR)** = observed arrests ÷ expected arrests. 1.00 = as expected.
4. 95% range = exact Poisson interval of the observed count, divided by expected.
5. p-value = 2 × min(P(X ≤ obs), P(X ≥ obs), 0.5) with X ~ Poisson(expected).
6. Correct the p-values for 22 districts with **Benjamini–Hochberg FDR**: sort p, multiply by n ÷ rank, enforce monotonic. Flag "above/below expected" only if corrected p < 0.05 *and* the range excludes 1.

Example: District 11 observed 38.7% vs expected 39.5% → index **0.98**. Districts with fewer than about 3,000 crimes are excluded from this analysis.

8.11 What predicts arrest (`arrest_models()`)

- **Target:** Arrest (yes/no). **Inputs:** crime type (top 25 + "other"), location (top 15 + "other"), district, time of day, weekday, domestic, weekend. No future information is used.
- **Split in time:** train on records before 2017-01-01 (**474,279**), test on records after (**153,508**), so the test is "does it work on new months?".
- **Logistic regression** (one-hot encoding, reference level = most common value, C = 100): odds ratios with 95% ranges from **15 full-size bootstrap resamples**. Test **AUC 0.837**. Odds ratios for near-100%-arrest types are marked unstable.
- **Gradient boosting** (200 trees, learning rate 0.1): test **AUC 0.865**.
- **Brier score** (lower is better): base-rate guess 0.162; boosting 0.095.
- **Permutation importance** (shuffle one input, see how much AUC drops; 5 repeats, 40,000 test rows): crime type 0.31, location 0.10, district 0.009, domestic 0.008, time of day 0.004, weekday 0.0.
- **Calibration:** test rows split into 10 groups by predicted chance, compared with the actual arrest share.

8.12 Domestic and location analysis

Domestic share = domestic ÷ all, by crime type and by hour (placeholders removed). Arrest rate for domestic vs non-domestic within each type. Location profile = crimes, arrest %, domestic % per Location Description (top 15).

8.13 Spatial metrics (`stats_geo.py`)

- **Grid (`grid_density()`):** latitude/longitude rounded into squares of 0.005° (about 550 m). Per square: crimes, arrests, violent, property, other, and its **dominant district** (the district code most common among its crimes). Squares with fewer than 5 crimes are dropped. Result: 2,387 squares.
- **District centroid:** the *median* latitude and longitude of that district's located crimes.
- **Concentration (`concentration()`):** sort the 275 police beats by crime count; Lorenz curve = cumulative crime share vs cumulative beat share; **Gini = 0.209** (0 = perfectly even, 1 = all in one beat); top 10% of beats hold **18.2%** of crimes, top 20% hold **31.7%**. Conclusion: spread out, not hot-spot dominated.
- **Location quotient (`location_quotients()`):** LQ = (district's share of crime type X) ÷ (city's share of crime type X). Above 1 = over-indexes. District 11 narcotics LQ = **3.74**.
- **Map colours (website):** each district's crimes, or arrest ÷ crimes, violent ÷ crimes, domestic ÷ crimes, night ÷ crimes, shown on a colour ramp. Rate rankings ignore districts below 0.15% of the crimes in view (tiny-sample noise, e.g. a district with only 31 records).

8.14 Helper statistics (`statlib.py`)

Function	What it computes
`wilson()`	Confidence range for a percentage
`poisson_ci()`	Exact range for a count (via chi-square quantiles)
`bh_fdr()`	Benjamini–Hochberg corrected p-values
`bootstrap_ratio()`	Range for mean(a) ÷ mean(b) by resampling
`cramers_v()`	Strength (0–1) of link between two categories
`clean_records()`	Turns tables into JSON-safe lists (no NaN)

9. How results become the website

1. **Insights** (`insights.py`): sentences filled from computed values, each with evidence, method, charts and caveat, tagged headline / supporting / data_quality.
2. **Export** (`export.py → write_all()`): `kpis.json`, one JSON per chart, `insights.json`, `methodology.json`, `cleaning_log.json`, map files, `manifest.json` (every file with its SHA-256), JSON Schemas and `types.ts`.
3. **The cube** (`pipeline.py → build_cube()`): all 1,048,575 rows grouped by (crime type, district, month, weekday, time of day, arrest, domestic) → **523,742 rows** of counts, stored as integer arrays with lookup tables (0.66 MB compressed).
4. **In the browser** (`engine.ts → Engine.compute()`): when you filter, one pass over those counts (typed arrays) rebuilds K1–K8 and Q1–Q10 together, so they always agree. `districtStats()` and `typeStats()` do the same for the map and for Dot.
5. **Dot** (`assistant.ts`, `dot.ts`): matches your question to a topic, runs the same live maths or reads the insights/methodology files, and never invents numbers.

10. How we proved the numbers are right

- **Handout anchors:** K1 = 1,048,575; arrests = 214,846 (20.5%); Q1 has 28 points with a Feb 2016 low and Aug highs.
- **Internal sums:** Q2, Q3, Q5, Q10 add to K1; Q4 adds to K5; Q5 shares add to 100%.
- **Orders and sorting:** Mon→Sun, Night→Evening, Winter→Fall; Q7/Q8/Q9 sorted high→low; sizes 10/5/10; Q10 has 28 cells.
- **Cube vs raw:** cube totals equal the raw rows, and **25 random type × district filter combinations** match exactly.
- **Website vs Python:** `npm run verify` compares every KPI and chart (engine output equals export) and the page repeats a 15-point self-check on load (the "Verified" badge).
- **Source untouched:** SHA-256, size and modified-time unchanged.
- **Shape of files:** Pydantic validates every exported file.
- All **44** Python checks pass in `python run_all.py`.

11. Limitations, in plain words

- The file is a **slice**; only two periods are complete, so we make **no long-term trend claims**.
- No population or area figures, so no "per person" rates.
- Window B has almost no coordinates, and its arrest rate is likely too low because arrests are recorded later.
- The "1st of the month" and "placeholder time" explanations are strong guesses, not proven facts.
- Models show **association, not cause**; the district index ignores clustering in space, so its ranges are optimistic.
- Dot is rule-based (not a free-form AI) and only knows this project.
- We only tested desktop screen sizes; the `pytest` wrapper was not run (not installed) although the same 44 checks run inside `run_all.py`.
